SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

NAME: ANSHIKA

REG. NO: 1924260252

COURSE: CSA0808- Python Programming

SLOT: D

GITHUB: https://github.com/anshiikaa22/CSA0808

TASK-1: Decisions with Many Boundaries

Aim: To implement a state electricity subsidy eligibility system using separate if statements and if-elif-else, test all important boundaries, and compare the decisions.

1. Prediction and Test Cases

Before running the program, the expected decision for eight households is predicted below. Full subsidy is given for 100 units or less. Partial subsidy is given for 101 to 200 units only when the household has a ration card. Above 200 units, special subsidy is given only when a member is above 80 years.2.

2.Python Program

```
def subsid _separate(units, ration, senior):
    results = []

    if units <= 100:
        results.append("Full Subsidy")

    if 101 <= units <= 200 and ration:
        results.append("Partial Subsidy")

    if units > 200 and senior:
        results.append("Special Subsidy")

    if len(results) == 0:
        results.append("No Subsidy")

    return results


def subsidy_chain(units, ration, senior):
    if units <= 100:
        return "Full Subsidy"
    elif 101 <= units <= 200 and ration:
        return "Partial Subsidy"
    elif units > 200 and senior:
        return "Special Subsidy"
    else:
        return "No Subsidy"


households = [
    (100, True, False),
    (101, True, False),
    (150, False, False),
    (200, True, False),
    (201, True, False),
    (250, False, True),
```

```
    (250, True, False),
    (80, False, True)
]

print("Units | Separate if | if-elif-else")
for units, ration, senior in households:
    print(units, "|", subsidy_separate(units, ration,
senior),
          "|", subsidy_chain(units, ration, senior))
```

3. Sample Output

```
Units | Separate if | if-elif-else
100 | ['Full Subsidy'] | Full Subsidy
101 | ['Partial Subsidy'] | Partial Subsidy
150 | ['No Subsidy'] | No Subsidy
200 | ['Partial Subsidy'] | Partial Subsidy
201 | ['No Subsidy'] | No Subsidy
250 | ['Special Subsidy'] | Special Subsidy
250 | ['No Subsidy'] | No Subsidy
80 | ['Full Subsidy'] | Full Subsidy
```

Result: The electricity subsidy program was implemented using separate if statements and if-elif-else. The boundary cases were tested, and if-elif-else was selected because it gives one clear decision and is easy to maintain.

TASK-2: Loops that Count Attempts

Aim: To implement OTP verification using while and for loops, demonstrate break, continue, pass and loop-else, and allow two extra attempts after a correct security answer.

1. Program Using while Loop

```
otp = "1234"
attempts = 0

while attempts < 3:
    entered = input("Enter OTP: ")

    if entered == "":
        print("Blank entry. Try again.")
        continue

    attempts += 1

    if entered == otp:
        print("OTP verified successfully")
        break
    else:
        print("Wrong OTP")

else:
    print("Account locked for 30 minutes")
```

2. Program Using for Loop

The inner while loop asks again after a blank entry. This ensures that a blank entry does not consume one of the three attempts.

```
otp = "1234"

for attempt in range(3):
    entered = ""

    while entered == "":
        entered = input("Enter OTP: ")
        if entered == "":
            print("Blank entry. Try again.")

    if entered == otp:
        print("OTP verified successfully")
        break
    else:
        print("Wrong OTP")
else:
    print("Account locked for 30 minutes")
```

3. Sample Outputs

```
Example 1 – Correct OTP
Enter OTP: 1234
OTP verified successfully

Example 2 – Three wrong attempts
Enter OTP: 1111
Wrong OTP
Enter OTP: 2222
Wrong OTP
Enter OTP: 3333
Wrong OTP
Account locked for 30 minutes

Example 3 – Blank entry
Enter OTP:
Blank entry. Try again.
Enter OTP: 1234
OTP verified successfully
```

4. Changed Condition – Two Extra Attempts

```
otp = "1234"
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    entered = input("Enter OTP: ")

    if entered == "":
        print("Blank entry. Try again.")
        continue

    attempts += 1

    if entered == otp:
        print("OTP verified successfully")
        break

    print("Wrong OTP")

    if attempts == 3:
        answer = input("Security question correct?
(yes/no): ")
        if answer.lower() == "yes":
            max_attempts = 5
            print("Two extra attempts allowed")
else:
    print("Account locked for 30 minutes")
```

Result: OTP verification was implemented using while and for loops. Blank entries are ignored, successful verification stops the loop, and the account is locked after all permitted attempts.

TASK-3: Vehicle Registration Number Validator

Aim: To normalise, validate and format vehicle registration numbers using Python string methods and slicing without regular expressions.

1. Prediction and Test Cases

Input	Normalised form	Prediction
tn 09 ab 1234	TN09AB1234	Valid
KA-01-M-0007	KA01M0007	Valid
TN9AB1234	TN9AB1234	Invalid
TN 22 1234 AB	TN221234AB	Invalid
MH 12 CD 5678	MH12CD5678	Valid
KL 07 A 1234	KL07A1234	Valid

2. Python Program

```
def normalise(text):
    text = text.strip()
    text = text.replace(" ", "")
    text = text.replace("-", "")
    return text.upper()


def is_valid(reg):
    if len(reg) not in (9, 10):
        return False

    state = reg[0:2]
    district = reg[2:4]
    series = reg[4:-4]
    number = reg[-4:]

    return (state.isalpha() and
            district.isdigit() and
            len(series) in (1, 2) and
            series.isalpha() and
            number.isdigit())


def format_number(reg):
    if not is_valid(reg):
        return "Invalid Number"

    return (reg[0:2] + " " + reg[2:4] + " " +
            reg[4:-4] + " " + reg[-4:])


numbers = [
    "tn 09 ab 1234",
    "KA-01-M-0007",
    "TN9AB1234",
    "TN 22 1234 AB",
```

```
    "MH 12 CD 5678",
    "KL 07 A 1234"
]

for raw in numbers:
    reg = normalise(raw)
    if is_valid(reg):
        print(raw, "->", reg, "Valid")
        print("Receipt:", format_number(reg))
    else:
        print(raw, "->", reg, "INVALID")
```

3. Sample Output

```
tn 09 ab 1234 -> TN09AB1234 Valid
Receipt: TN 09 AB 1234
KA-01-M-0007 -> KA01M0007 Valid
Receipt: KA 01 M 0007
TN9AB1234 -> TN9AB1234 INVALID
TN 22 1234 AB -> TN221234AB INVALID
MH 12 CD 5678 -> MH12CD5678 Valid
Receipt: MH 12 CD 5678
KL 07 A 1234 -> KL07A1234 Valid
Receipt: KL 07 A 1234
```

Result: The registration number was normalised, validated and formatted successfully using string methods and slicing. Invalid inputs were rejected.

TASK-4: Building Long Text: + or join()?

(6 MARKS)

Aim: To build a long monthly statement using repeated +=, str.join() and f-strings, and compare their execution times.

1. Prediction

The join() method is expected to perform well for large text because it joins the string parts in one operation. Repeated += may repeatedly create new strings. F-strings are useful for formatting each line; they can be combined with join().

2. Python Program

```
import time

items = [f"Item{i}:Rs{i % 50}" for i in
range(200_000)]


def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text


def build_join(parts):
    return "".join(p + ";" for p in parts)


def build_fstring(parts):
    lines = [f"{p};" for p in parts]
    return "".join(lines)


for size in [1000, 10000, 200000]:
    data = items[:size]
    print("\\nNumber of lines:", size)

    for function in [build_plus, build_join,
build_fstring]:
        times = []
        for run in range(3):
            start = time.perf_counter()
            result = function(data)
            elapsed = time.perf_counter() - start
            times.append(elapsed)

        average = sum(times) / len(times)
```

```
        print(function.__name__,
              "Average:", round(average, 4),
"seconds")

# Check that all methods produce the same text
a = build_plus(items)
b = build_join(items)
c = build_fstring(items)
print("Same text:", a == b == c)
print("Length:", len(a))
```

3. Sample Output

Execution times depend on the computer and Python version. Run the program three times for each size and record the displayed average. The text length for 200,000 items should be 3,048,890 characters.

```
Number of lines: 1000
build_plus Average: [measured time] seconds
build_join Average: [measured time] seconds
build_fstring Average: [measured time] seconds

Number of lines: 10000
build_plus Average: [measured time] seconds
build_join Average: [measured time] seconds
build_fstring Average: [measured time] seconds

Number of lines: 200000
build_plus Average: [measured time] seconds
build_join Average: [measured time] seconds
build_fstring Average: [measured time] seconds

Same text: True
Length: 3048890
```

Result: The three string-building methods were implemented. join() is recommended for combining many string parts, while file writing or StringIO can be considered for very large statements.

TASK-5: Functions, Parameters and Scope

(7 MARKS)

Aim: To identify the mutable default argument problem and implement independent customer orders using functions, parameters, return values and variale scope.

1. Python Program

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order


def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order


print("Buggy function:")
print(add_item_buggy("Tea"))
print(add_item_buggy("Vada"))

print("Corrected function:")
print(add_item("Tea"))
print(add_item("Vada"))

print("Default values:")
print(add_item_buggy.__defaults__)


# Returning more than one value
def calculate_bill(amount):
    gst = amount * 0.05
    total = amount + gst
    return total, gst


total, gst = calculate_bill(1000)
print("Total:", total)
print("GST:", gst)
```

```
# Local and global scope
x = 10

def show_scope():
    y = 20
    print("Global:", x)
    print("Local:", y)

show_scope()


# Modifying a global variable
count = 0

def increase():
    global count
    count += 1

increase()
print("Updated global:", count)


# Nonlocal scope
def outer():
    value = 10

    def inner():
        nonlocal value
        value = 20

    inner()
    print("Nonlocal:", value)

outer()
```

2. Sample Output

```
Buggy function:
['Tea']
['Tea', 'Vada']
Corrected function:
['Tea']
['Vada']
Default values:
(['Tea', 'Vada'],)
Total: 1050.0
GST: 50.0
Global: 10
Local: 20
Updated global: 1
Nonlocal: 20
```

3. Three Independent Customer Orders

```
customer1 = add_item("Tea")
customer2 = add_item("Vada")
customer3 = add_item("Coffee")

print(customer1)
print(customer2)
print(customer3)
```

Expected output:

```
['Tea']
['Vada']
['Coffee']
```

Result: The mutable default argument bug was demonstrated and corrected using None. The corrected design creates independent orders for different customers.

TASK-6: Recursion and Its Limits

(8 MARKS)

Aim: To calculate the number of ways to climb temple steps using normal recursion, memoisation and iteration, and compare their calls, time and space complexity.

1. Python Program

```
from functools import lru_cache
import sys

calls = 0


def ways_naive(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n - 1) + ways_naive(n - 2)


@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_memo(n - 1) + ways_memo(n - 2)


print("Answers from 1 to 6:")
for n in range(1, 7):
    ways_memo.cache_clear()
    print(n, "steps:", ways_memo(n))


print("\\nCall count comparison:")
for n in [10, 20, 25, 30]:
    calls = 0
```

```
    print("Naive:", n, "=", ways_naive(n), "calls
=", calls)

    ways_memo.cache_clear()
    calls = 0
    print("Memo :", n, "=", ways_memo(n), "calls =",
calls)


def ways_loop(n):
    if n <= 1:
        return 1

    a, b = 1, 1
    for i in range(2, n + 1):
        a, b = b, a + b
    return b


print("\\nLoop result for 1000 steps:")
print(ways_loop(1000))

print("Recursion limit:", sys.getrecursionlimit())
try:
    ways_memo.cache_clear()
    print("Memoised result for 1000:",
ways_memo(1000))
except RecursionError:
    print("Recursion limit exceeded")
```

2. Answers for n = 1 to 6

Number of steps	Number of ways
1	1
2	2
3	3
4	5
5	8
6	13

Result: The staircase problem was solved using recursion, memoisation and iteration. Memoisation reduces repeated calculations, while iteration is the safest choice for 1,000 steps because it avoids recursion-depth errors and uses constant auxiliary space.

TASK-7: Diagnose, Redesign and Validate

(13 MARKS)

Aim: To identify a performance limitation in the original staircase program, redesign it using an iterative method, and compare the original and redesigned solutions.

A. Diagnose the Bottleneck – 2 Marks

The selected program is Q6, the normal recursive staircase counter. Its main bottleneck is repeated calculation of the same smaller staircase values. For example, the normal recursive version makes 242,785 calls for n = 25 and 2,692,537 calls for n = 30. The number of calls grows rapidly as the input increases.

B. Define the Design Objective – 2 Marks

The objective is to calculate the same number of staircase combinations for inputs up to 1,000 steps while reducing the running time and using constant auxiliary space.

C. Design the Revised Solution – 4 Marks

The redesigned program uses iteration and stores only the previous two results. It does not need a recursion stack or a dictionary cache.

Pseudocode

```
START
INPUT n
IF n <= 1
    RETURN 1
SET a = 1, b = 1
FOR i FROM 2 TO n
    SET a, b = b, a + b
RETURN b
STOP
```

Redesigned Python Program

```
def ways_original(n):
    if n <= 1:
        return 1
    return ways_original(n - 1) + ways_original(n -
2)


def ways_redesigned(n):
    if n <= 1:
        return 1

    a, b = 1, 1
    for i in range(2, n + 1):
        a, b = b, a + b

    return b


for n in [10, 20, 25, 30]:
    print("Steps:", n)
    print("Original:", ways_original(n))
    print("Redesigned:", ways_redesigned(n))
    print("Same result:",
          ways_original(n) == ways_redesigned(n))


print("1000 steps:", ways_redesigned(1000))
```

D. Prediction Before Implementation – 1 Mark

The redesigned program is expected to produce the same answer as the original program for every tested input. It is expected to reduce the time from exponential growth to linear growth and reduce auxiliary space to O(1).

E. Implement and Validate – 2 Marks

Steps	Original result	Redesigned result	Validation
10	89	89	Same
20	10,946	10,946	Same
25	121,393	121,393	Same
30	1,346,269	1,346,269	Same
1000	Very large integer	Very large integer	Same recurrence

For an exact timing comparison, run both functions with the same input and use time.perf_counter(). The original recursive function is practical for smaller values such as 10 to 30. The redesigned loop should be used for 1,000 steps.

```
import time

for n in [10, 20, 25, 30]:
    start = time.perf_counter()
    original = ways_original(n)
    original_time = time.perf_counter() - start

    start = time.perf_counter()
    redesigned = ways_redesigned(n)
    redesigned_time = time.perf_counter() - start

    print("Steps:", n)
    print("Original time:", original_time)
    print("Redesigned time:", redesigned_time)
    print("Same result:", original == redesigned)
```

Result: The staircase counter was successfully redesigned using iteration. The redesigned program gives the same results as the original for the tested inputs, avoids recursion-depth errors, and uses less auxiliary memory.